

ASANSOL ENGINEERING COLLEGE • ALUMNI TALK

The Mathematics Behind Visual Language Models

Linear algebra, calculus, and optimization — from your 2nd-year syllabus to GPT-4V

Subhabrata Bhattacharya • B.Tech CSE, Class of 2003
Machine Learning & Computer Vision

60-minute session • Talk 1 of 3: Math → Build → Deploy

Why this talk?

1

VLMs look like magic

who's used ChatGPT, uploaded an image, and wondered what's happening inside?

2

Underneath: math you already study

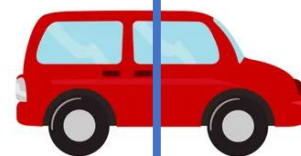
Linear algebra + multivariable calculus + probability — applied at massive scale, not replaced by something new.

3

Goal for today

In 2003 I sat where you sit. The math didn't change — let's trace it through one red car.

This talk: the math. Talks 2–3: building and deploying a VLM yourself.



One red car

Patch 1

Patch 2

What is a Visual Language Model, structurally?



Core idea to hold onto:

Everything is a tensor transformation. Training = adjusting the numbers inside matrices so outputs match reality.

Vectors & Vector Spaces

- **Embeddings are just points (vectors) in a high-dimensional space**
- **Dimension = capacity to represent distinct shades of meaning**
- **Cosine similarity / dot product = a numeric measure of “meaning closeness”**



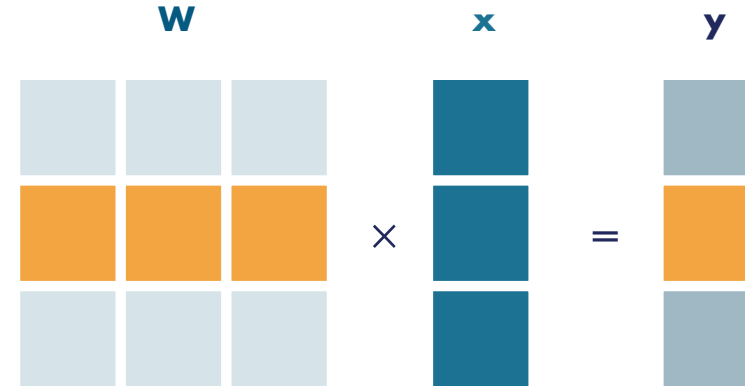
Small angle = related meaning. Large angle = unrelated.

Matrices as Transformations

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

The single most repeated operation in deep learning

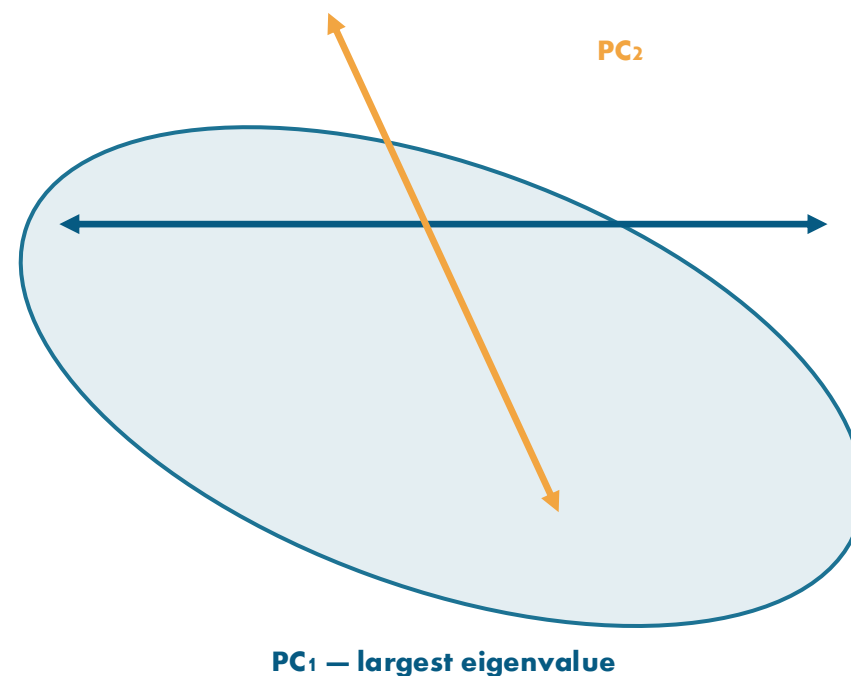
- **Matrix multiplication = a weighted combination = “asking questions of the data”**
- **W (the weight matrix) is what training actually adjusts**
- **Rank of a matrix: how much independent information it carries**
- **Low-rank matters later — preview of LoRA fine-tuning**



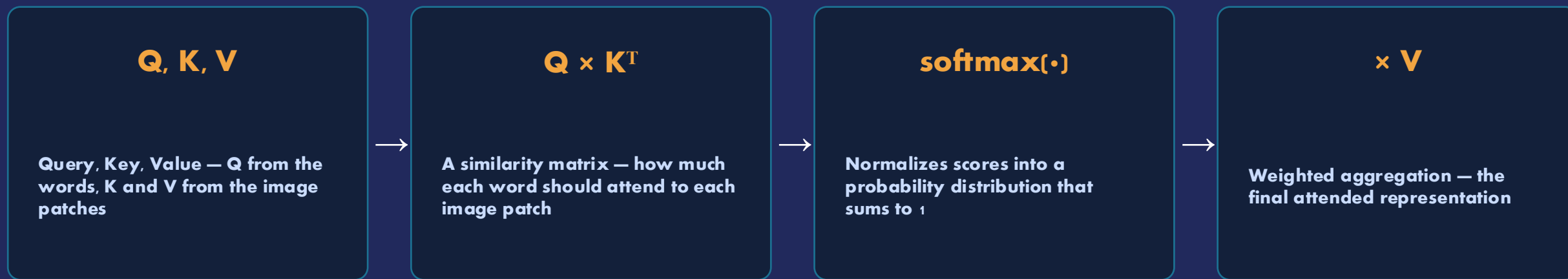
Each output value = a weighted sum across the input vector

Eigenvalues, Eigenvectors & PCA Intuition

- **Eigenvectors:** directions a transformation stretches or compresses, without rotating
- **Eigenvalues:** how much stretching happens along that direction
- **PCA** uses this to find the directions of greatest variance in data
- **In VLMs:** helps us understand what an embedding space actually “cares about” — which directions carry the most signal



Attention as Linear Algebra



Cross-attention: the word “car” asks a question of every image patch — which of you matters to me? At its core, attention is linear algebra + normalization.

From Images to Vectors: Vision Encoding

- **Convolution** = structured matrix multiplication with weight sharing and local receptive fields
- **ViT-style patching**: image $\rightarrow$ grid of patches $\rightarrow$ each patch flattened & linearly projected into a vector
- **Positional encoding**: matrices alone lose spatial order — we re-inject “where” explicitly
- **CLIP-style contrastive pretraining** aligns image and text vector spaces before VLM training even begins

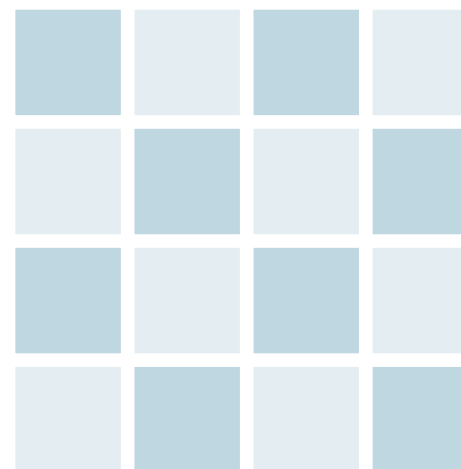


image $\rightarrow$ patch grid

$\downarrow$ *flatten + project*

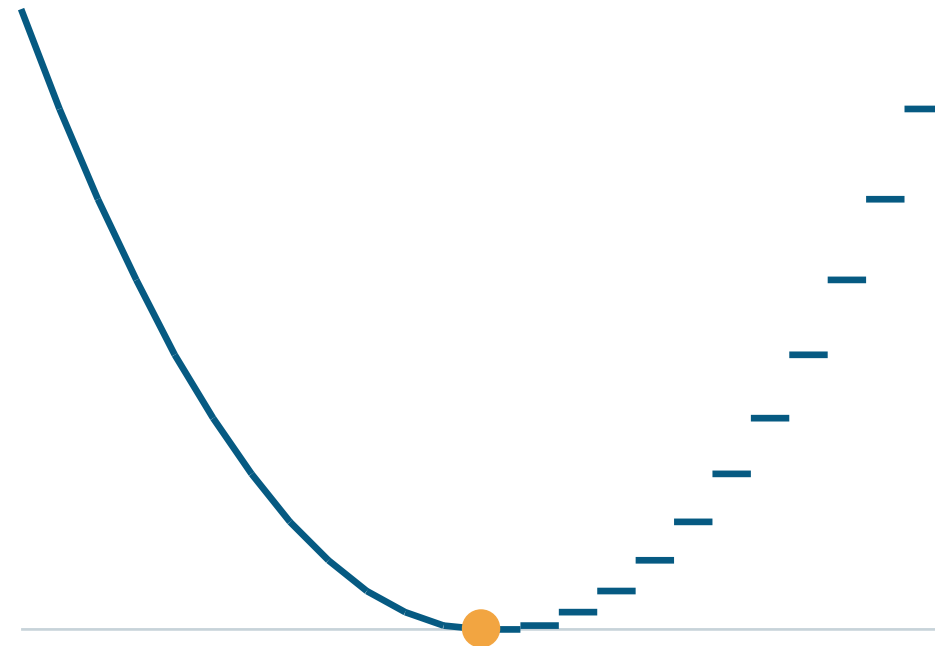


sequence of patch vectors

The Learning Problem as Function Minimization

minimize $L(w)$

- **Loss function L :** a numeric distance between prediction and ground truth
- **w = every trainable number in the model** (all the W matrices, all the biases)
- **Training = searching for w that minimizes L , averaged over the data distribution**

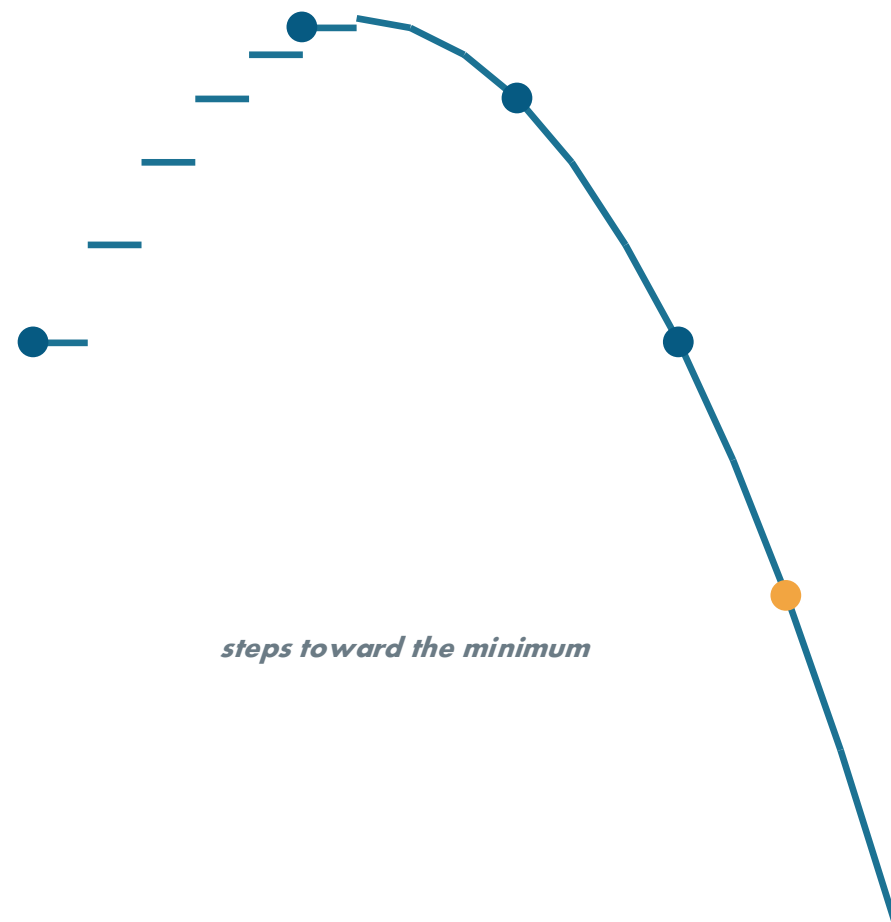


minimum = the trained model

Gradient Descent & Calculus Recap

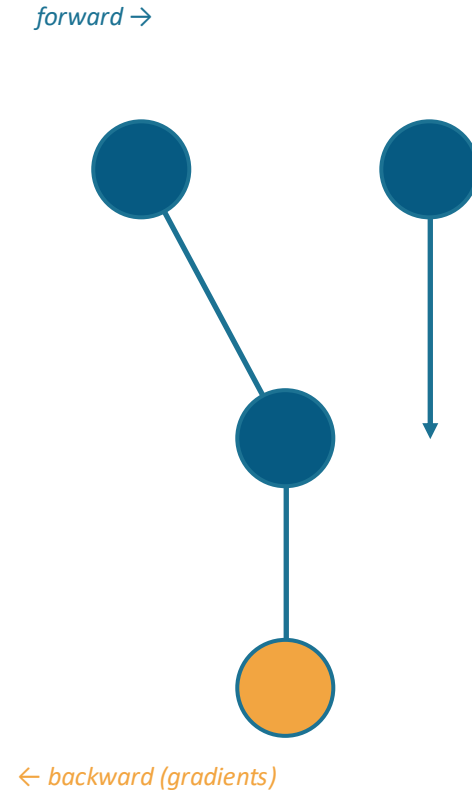
$$\mathbf{w} \leftarrow \mathbf{w} - \text{lr} \cdot \nabla L(\mathbf{w})$$

- **Derivative:** direction of steepest change (1 variable)
- **Gradient ∇L :** the multivariable extension — one partial derivative per parameter
- **lr = learning rate:** how big a step to take
- **Too high** → overshoots and diverges. **Too low** → crawls, may never converge in time



Backpropagation

- The chain rule, applied through a computation graph
- Every layer's gradient depends on the gradient of the layer after it
- Gradients themselves are matrices / tensors — this is matrix calculus in practice
- Vanishing / exploding gradients: why very deep networks are hard to train directly
- → motivates architecture choices: residual connections, normalization layers



Beyond Vanilla SGD

Momentum

Keeps a moving average of past gradients — smooths the path, powers through small bumps

Adam Optimizer

Tracks both the average gradient and its variance — adapts the step size per parameter

Loss Landscapes

High-dimensional space is mostly saddle points, not bad local minima — this is why Momentum and Adam keep working at scale

Putting It Together: Training a VLM

Stage 1

Vision-Language Alignment

Contrastive or projection-only pretraining — teach the model that a picture of a dog and the word “dog” should land near each other



Stage 2

Instruction Fine-Tuning

Supervised training on image + text pairs — teach the model to follow instructions about images



Stage 3

Preference Optimization

RLHF-style methods — the optimization objective shifts from “match the label” to “match human preference”

Efficiency Tricks, Traced Back to the Math

LoRA

*Low-Rank
Adaptation*

Update W with a low-rank product $A \cdot B$ instead of the full matrix — e.g., a 4096×4096 matrix (16.8M values) becomes two rank-8 matrices (~65K values), a 250x reduction

Quantization

*Numerical
Precision*

Store weights in fewer bits — a storage/precision tradeoff on the same underlying linear algebra

Mixed Precision

Training Speed

Use lower precision for most compute, higher precision where gradients need it — balances speed and stability

Vectors & Attention, in Code

```
import numpy as np

# Slide 4: cosine similarity = "meaning closeness"
cat, kitten = np.array([0.9, 0.1]), np.array([0.85, 0.15])
airplane = np.array([0.1, 0.9])
cos_sim = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos_sim(cat, kitten))      # ~0.99 -> related
print(cos_sim(cat, airplane))   # ~0.35 -> unrelated

# Slide 7: attention in 4 lines
def attention(Q, K, V):
    scores = Q @ K.T / np.sqrt(K.shape[-1])
    weights = np.exp(scores) / np.exp(scores).sum(-1, keepdims=True)
    return weights @ V
```

Gradient Descent, in Code

```
import numpy as np

# toy loss, minimum at w = 3 (Slide 9 & 10)
L = lambda w: (w - 3) ** 2
grad = lambda w: 2 * (w - 3)

w, lr = 0.0, 0.1
for step in range(20):
    w = w - lr * grad(w)

print(w)  # converges toward 3.0
```

One line of math from Slide 10 ($w \leftarrow w - lr \cdot \text{grad } L(w)$) becomes one line of code.

Autograd: Backprop Without the Pain

```
import torch

w = torch.tensor(0.0, requires_grad=True)
loss = (w - 3) ** 2
loss.backward()

print(w.grad)    # -6.0, via the chain rule
                 # computed automatically
```

The chain rule from Slide 11 is one call: `.backward()`. This is what every training loop does.

LoRA: Same Idea, Two Lines

```
import torch

W = torch.randn(4096, 4096)    # 16.8M values, frozen
A = torch.randn(4096, 8, requires_grad=True) # ~33K trainable
B = torch.randn(8, 4096, requires_grad=True) # ~33K trainable

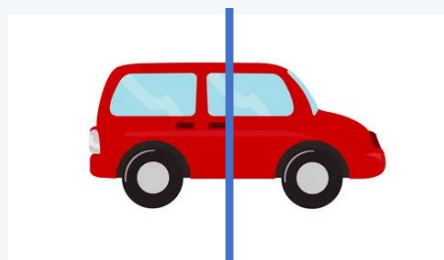
W_adapted = W + A @ B    # only A, B get updated in fine-tuning
```

Closes the loop with Slide 14 — same 250x reduction, now visible as two small tensors.

INTERACTIVE

Quick Check: Is This Image Related to This Caption?

"If I show you an image and a caption, what's the simplest linear algebra check that they're related?"



Patch 1

Patch 2

One red car

Mini walkthrough on the board:

- A 3-word sentence and a 2-patch image, each turned into vectors
- Compute Q , K , V for all five tokens
- One attention step: which patch does each word look at most?

<https://github.com/subharya83/cvml-exercise/tree/main/ex31>

CLOSING

The math doesn't change. The frameworks do.

- Linear algebra, multivariable calculus, and probability underlie VLMs — and robotics, recommendation systems, and scientific computing too
- Strengthen these fundamentals now; specific tools and libraries will keep changing around you
- From this campus in 2003 to this exact kind of work today — the path runs straight through the math you're studying right now

Thank you — Questions?